



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Code of Conduct:

*I **THULASI KRISHNA K (192421097)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1. Testing as an Engineering Activity

Overview

Software testing is not simply the process of executing a few test cases after development. It is an engineering activity that requires planning, analysis, systematic test design, controlled execution, measurement, and continuous improvement. In a banking application, this approach is particularly important because failures can affect financial transactions, account information, and customer trust.

A suitable testing approach should begin with a detailed study of the requirements. Each requirement should be converted into identifiable test conditions, and the conditions should then be transformed into test cases. The expected result must be defined before execution so that the actual behavior can be compared objectively.

Proposed Testing Approach

The banking team can redesign its testing process using the following flow:

Requirement Analysis → Risk Identification → Test Planning → Test Design → Test Execution → Defect Reporting → Retesting → Regression Testing

During requirement analysis, testers should identify functional and non-functional requirements. Critical functions such as login, account balance, fund transfer, withdrawal, and transaction history should receive higher priority.

Test cases should be maintained using a test management system, while defects should be recorded in a centralized defect-tracking system. Automated testing can be introduced for repetitive regression scenarios.

Suitable Test Design Techniques

The following techniques can replace ad-hoc testing:

1. **Equivalence Partitioning** – Divides input data into valid and invalid classes.
2. **Boundary Value Analysis** – Tests values at the limits and around the limits.
3. **Decision Table Testing** – Tests different combinations of conditions and actions.
4. **State Transition Testing** – Verifies changes between different transaction states.
5. **Risk-Based Testing** – Prioritizes high-risk banking operations.
6. **Regression Testing** – Ensures that changes do not introduce new defects.

Practical Illustration

Consider a fund-transfer feature. Instead of testing only a successful transaction, the tester should consider:

- Valid transfer amount
- Insufficient balance
- Invalid beneficiary
- Maximum transaction limit
- Duplicate transaction
- Network interruption

- Transaction timeout
- Unauthorized transaction

For example, if ₹10,000 is transferred from an account containing ₹25,000, the system should deduct exactly ₹10,000 and update the balance to ₹15,000. The transaction should also appear correctly in the transaction history.

Quality Improvement

A systematic engineering approach improves:

- Defect detection
- Requirement coverage
- Test repeatability
- Defect traceability
- Regression effectiveness
- Overall reliability

Conclusion: Treating testing as an engineering activity changes testing from random checking into a planned quality-assurance process. Formal test techniques, tools, standards, and risk-based prioritization can significantly reduce defects in banking applications.

2. Structured Testing Process Across the SDLC

Testing Across Development Stages

An ATM system requires a clearly defined testing process because it handles authentication, account information, cash dispensing, and financial transactions. A structured process should be applied throughout the Software Development Life Cycle rather than only after coding is completed.

Requirement and Planning Stage

The testing team should first examine requirements related to:

- Card validation
- PIN authentication
- Account verification
- Balance enquiry
- Cash withdrawal
- Transaction limits
- Receipt generation
- Error handling

At this stage, testers can identify ambiguous or missing requirements before they become defects.

Test Design and Implementation

Once requirements are finalized, appropriate test cases should be prepared. Both positive and negative scenarios must be included.

For ATM withdrawal, test cases may include:

Test Condition	Expected Result
Valid card and PIN	Transaction proceeds

Test Condition	Expected Result
Invalid PIN	Transaction rejected
Insufficient balance	Withdrawal rejected
Amount above limit	Transaction rejected
Invalid amount	Appropriate error displayed
ATM has insufficient cash	Transaction handled safely
Network failure	Transaction should not cause incorrect deduction

Execution and Defect Management

During execution, actual results are compared with expected results. Any mismatch is recorded as a defect. After the defect is fixed, retesting confirms the correction, followed by regression testing to ensure that existing functionality has not been affected.

Importance of Process Consistency

A consistent process provides:

- Repeatable testing
- Better requirement coverage
- Easier defect tracking
- Clear responsibilities
- Reduced testing gaps
- Improved release confidence

Expected Outcome

If every SDLC stage includes suitable testing activities, defects can be detected earlier. For example, an unclear withdrawal limit can be identified during requirement review instead of being discovered after deployment.

Conclusion: A structured SDLC testing process provides systematic quality control from requirements through release. For ATM systems, it improves transaction accuracy, security, reliability, and user confidence.

3. Errors, Defects and Failures

Understanding the Three Terms

Errors, defects, and failures describe different stages of a software problem. Correctly distinguishing these concepts is essential for effective testing and defect reporting.

An **error** is a human mistake made during requirement analysis, design, coding, or another development activity. A **defect** is the flaw introduced into the software as a result of that error. A **failure** is the incorrect behavior observed when the defective software is executed.

Relationship

The relationship can be represented as:

Human Error → Software Defect → Execution → Failure

For example, suppose a developer misunderstands a requirement stating that a password must contain at least eight characters. The developer implements a minimum length of five characters.

In this situation:

- The misunderstanding is the **error**.
- The incorrect validation logic is the **defect**.
- Allowing a five-character password when eight are required is the **failure**.

Login and Transaction Example

Consider a transaction application where the expected transfer amount is ₹5,000 but the application deducts ₹6,000.

The tester observes the incorrect deduction and reports it as a failure caused by an underlying defect in transaction processing.

Effect on Defect Reporting

If these terms are misunderstood, defect reports may be vague. A statement such as “**transaction is wrong**” does not provide enough information.

A professional report should include:

1. Defect title
2. Environment
3. Preconditions
4. Test data
5. Steps to reproduce
6. Expected result
7. Actual result
8. Severity and priority
9. Logs or screenshots

Importance

Clear terminology helps developers and testers communicate effectively. It also makes defect analysis and root-cause investigation easier.

Conclusion: Errors originate from human mistakes, defects represent flaws in the software, and failures are the observable incorrect results. Understanding this relationship improves defect reporting and supports more effective software testing.

4. Defect Clustering and Test Prioritization

Concept

Defect clustering is an important testing principle which indicates that a relatively small number of modules often contain a large proportion of defects. This observation can be used to prioritize testing resources.

Identifying Defect-Prone Modules

Historical defect information should be examined to identify modules with:

- High defect frequency
- High severity defects
- Complex logic
- Frequent changes
- Numerous dependencies
- High customer usage

For example, in an online banking system, fund transfer may contain more defects than a simple profile-display module because it involves complex validation and multiple integrations.

Prioritizing Test Cases

Testing should not give equal priority to every test case. High-risk functions should be tested earlier and more thoroughly.

A suitable priority structure is:

Priority 1 – Critical: Payment, fund transfer, authentication

Priority 2 – Important: Account statement, notifications

Priority 3 – Low Risk: Cosmetic or rarely used features

Practical Example

Suppose historical data shows that 60% of serious defects occurred in the payment module. The testing team should increase test coverage for:

- Successful payments
- Failed payments
- Duplicate payments
- Payment timeout
- Network interruption
- Invalid card details
- Refund processing

Benefits

Applying defect clustering helps to:

- Focus limited testing resources
- Detect serious defects earlier
- Improve regression coverage
- Reduce testing effort in low-risk areas
- Increase overall testing efficiency

However, modules with fewer historical defects should still receive basic testing because defect concentration does not mean other modules are defect-free.

Conclusion: Defect clustering provides valuable information for test planning. Combining historical defect data with risk, complexity, and business importance allows the testing team to prioritize effectively and improve defect detection.

5. Defect Classification and Repository Utilization

Purpose of Classification

A defect repository is useful only when the information stored in it is properly classified and analyzed. If a company repeatedly experiences the same defects across releases, it indicates that historical defect information is not being converted into preventive testing knowledge.

Major Defect Categories

Defects can be categorized as:

- Requirement defects
- Design defects
- Coding defects
- Integration defects
- Interface defects
- Data defects
- Performance defects
- Security defects

Classification helps determine where and why defects are originating.

Using Repository Data

A useful defect record should contain:

Defect ID | Module | Description | Severity | Priority | Root Cause | Status | Resolution

Historical information can then be analyzed to identify patterns.

For example, if several releases contain repeated payment-calculation defects, the testing team can identify payment processing as a defect-prone area.

Preventing Recurrence

The following actions should be taken:

1. Perform root-cause analysis.
2. Convert important defects into regression test cases.
3. Improve requirement reviews.
4. Increase testing for defect-prone modules.
5. Introduce automation for repetitive tests.
6. Conduct code and design reviews.

Expected Improvement

The repository becomes more than a record of old problems. It becomes a **knowledge base for future testing**.

Conclusion: Proper defect classification and repository analysis help organizations identify recurring problems, improve test design, and prevent similar defects from appearing in future releases.

6. Mobile Banking Application – Fund Transfer Failures

Problem Analysis

Frequent crashes during fund transfer indicate that the testing process may not be covering sufficient combinations of transaction, device, network, and application conditions. Since fund transfer is a critical banking function, systematic investigation is required.

Initial Investigation

The tester should collect information about:

- Device model
- Operating-system version
- Application version
- Network condition
- Account balance
- Transfer amount
- Beneficiary details
- Exact steps before the crash
- Error messages
- Application and server logs

The same scenario should be reproduced several times to determine whether the failure is consistent.

Testing Areas

Functional Testing:

Verify normal and abnormal transfer operations.

Integration Testing:

Check communication between the mobile interface, authentication service, transaction service, and database.

Network Testing:

Test weak networks, disconnections, reconnections, and timeouts.

Performance Testing:

Check behavior during multiple simultaneous transactions.

Regression Testing:

Ensure that fixes do not affect previously working functions.

Example

Suppose the application crashes when a user presses “Transfer” while the network changes from strong to weak. The tester should reproduce this condition and analyze application logs.

The cause could be:

- Poor exception handling
- Timeout handling error
- Incorrect transaction state
- API communication problem

Improvements

The organization should introduce:

- Automated regression tests
- Network-condition testing
- Crash monitoring

- Device compatibility testing
- Exception-handling tests
- Transaction recovery testing

Conclusion: Systematic testing helps identify the actual conditions responsible for mobile banking crashes. Combining functional, integration, network, performance, and regression testing can significantly improve the reliability of fund-transfer operations.

7. Testing and Debugging

Fundamental Difference

Testing and debugging are connected but different activities.

Testing determines whether the software behaves as expected and identifies failures.

Debugging investigates the cause of a failure and modifies the software to remove the defect.

Comparison

Testing	Debugging
Detects failures	Finds causes
Usually performed by testers	Mainly performed by developers
Uses test cases	Uses code analysis, logs and debugging tools
Produces defect reports	Produces corrections
Verifies expected behavior	Investigates internal behavior

Example

Suppose an ATM deducts ₹5,000 but dispenses only ₹4,000.

The tester identifies the mismatch and reports it. The developer then investigates the cash-dispensing logic, transaction calculation, or hardware communication.

After the correction:

Retesting → Regression Testing

Problems When Confused

Confusing the two activities may result in:

- Poor defect ownership
- Incomplete investigation
- Unclear communication
- Repeated defects
- Insufficient regression testing

Recommended Approach

A structured workflow should be followed:

Test → Detect Failure → Report Defect → Debug → Fix → Retest → Regression Test

Conclusion: Testing and debugging should work together while maintaining clear responsibilities. Testing establishes that a problem exists, whereas debugging identifies and removes its cause.

8. Cross-Browser Testing for an E-Commerce Platform

Need for Cross-Browser Testing

An e-commerce website may be accessed using different browsers, versions, operating systems, and devices. Differences in browser rendering engines, JavaScript behavior, CSS support, and screen sizes can result in inconsistent behavior.

Testing Strategy

The testing team should first prepare a browser compatibility matrix covering the browsers and versions officially supported by the application.

Important functions include:

- Login
- Product search
- Product display
- Shopping cart
- Checkout
- Payment
- Order confirmation
- User profile

Defect Sources

Cross-browser defects may occur because of:

1. JavaScript incompatibility
2. CSS rendering differences
3. Browser-specific features
4. Responsive-design problems
5. Third-party component behavior
6. Different browser versions

Practical Example

Suppose the checkout button works correctly in one browser but does not respond in another.

The tester should:

- Reproduce the issue
- Record browser and version
- Check console errors
- Inspect network requests
- Compare behavior with supported browsers
- Record screenshots and evidence

Coverage Improvement

The team can use:

- Automated browser testing
- Compatibility matrices
- Responsive testing
- Manual exploratory testing
- Regression testing
- Browser-based test environments

Conclusion: Cross-browser testing ensures that users receive consistent functionality across supported environments. A combination of automation, manual testing, compatibility analysis, and regression testing provides stronger coverage.

9. Defect Classification in Payroll Systems

Nature of the Problem

Incorrect salary calculations caused by faulty program logic represent primarily a **coding or logical defect**. However, testers should also determine whether the coding problem originated from an incorrect requirement or design decision.

Classification

Potential defect sources include:

Requirement Level: Salary rules may be incorrectly specified.

Design Level: The calculation model may be incorrect.

Coding Level: The implemented formula may contain logical errors.

Data Level: Incorrect salary or deduction data may be processed.

Testing Techniques

Several techniques are suitable:

Equivalence Partitioning

Employees can be divided into categories according to salary or employment conditions.

Boundary Value Analysis

Salary limits, tax limits, and deduction limits should be tested around their boundaries.

Decision Tables

Useful when salary depends on multiple conditions.

Regression Testing

Ensures that a correction does not affect existing calculations.

Example

Suppose:

Basic Salary = ₹30,000

Allowances = ₹5,000

Deductions = ₹3,000

Expected result:

Net Salary = ₹32,000

If the application calculates ₹28,000, the tester should investigate the calculation logic and identify the underlying defect.

Prevention

Future defects can be reduced through:

- Requirement reviews
- Code reviews
- Unit testing
- Boundary testing
- Automated calculation tests
- Regression testing

Conclusion: Correct classification helps testers choose suitable testing techniques. A combination of requirement validation, calculation testing, boundary analysis, and regression testing improves payroll accuracy.

10. Structured Test Case Design

Need for Structured Design

Random testing may detect obvious failures, but it cannot provide confidence that important requirements have been adequately tested. Test cases should therefore be designed systematically before execution.

Components of a Good Test Case

A professional test case should contain:

1. Test Case ID
2. Objective
3. Preconditions
4. Input data
5. Test steps
6. Expected result
7. Actual result
8. Pass/Fail status
9. Defect reference, if required

Test Design Techniques

The following techniques can improve effectiveness:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Error Guessing
- Risk-Based Testing

Example: Login Function

Instead of randomly entering usernames and passwords, planned test cases should include:

Condition	Expected Behavior
Valid username + valid password	Login successful
Invalid username	Login rejected
Invalid password	Login rejected
Empty fields	Validation message
Locked account	Access denied
Repeated invalid attempts	Appropriate security response

Advantages

Structured test case design provides:

- Better requirement coverage
- Repeatable execution
- Clear expected results
- Easier defect identification
- Better regression testing
- Improved traceability

Conclusion: Structured test case design converts testing from random execution into a planned engineering activity. It increases defect detection and provides measurable evidence of software quality.

11. Data Integrity in Healthcare Applications

Nature of the Problem

Healthcare applications often consist of interconnected modules such as patient registration, appointments, laboratory services, pharmacy, billing, and medical records. Data inconsistency between these modules can reduce system reliability.

Possible Sources

Data defects may originate from:

- Incorrect requirements
- Database design problems
- Incorrect field mapping
- Duplicate records
- Integration errors
- Incorrect update logic
- Data transformation problems

Systematic Validation

A patient's information should be followed across the complete workflow:

Registration → Appointment → Laboratory → Pharmacy → Billing

Important information such as patient ID, name, date of birth, test information, and billing details should remain accurate wherever applicable.

Example

Suppose a patient's date of birth is correctly entered during registration but appears differently in the billing module.

The tester should determine whether the problem occurred during:

1. Data storage
2. Data transfer
3. Database retrieval
4. Data conversion
5. User-interface display

Improvements

The team should use:

- Integration testing
- Database validation
- Data consistency checks
- Regression testing
- Detailed defect reports
- Requirement traceability

Conclusion: Systematic data-integrity testing ensures that information remains accurate as it moves between healthcare modules. Proper integration and defect tracking are essential for maintaining reliable healthcare software.

12. Defect Repository and Recurring Defects

Purpose of a Defect Repository

Repeated defects indicate that historical testing information is not being effectively used. A defect repository can help the team identify patterns and convert previous failures into future preventive test cases.

Information to Record

Each record should include:

Defect ID | Module | Description | Severity | Priority | Root Cause | Status | Resolution

This information allows the organization to analyze defects across releases.

Identifying Patterns

The team can examine:

- Defects by module
- Defects by severity

- Defects by release
- Defects by root cause
- Frequently recurring defect types

For example, if discount-related defects repeatedly occur in an e-commerce system, additional tests should be created for different discount combinations.

Preventive Strategy

Important historical defects should be added to the regression suite.
The team should also:

1. Perform root-cause analysis.
2. Improve requirement reviews.
3. Increase testing in defect-prone modules.
4. Review previous defects before new releases.
5. Automate important regression scenarios.

Conclusion: A defect repository should be used as a source of testing knowledge rather than simply as a storage location. Analyzing historical patterns helps prevent recurring defects and improves future releases.

13. Peak-Usage Failures in a Web-Based Learning System

Problem Analysis

A learning platform may operate correctly with a small number of users but fail during examinations, result publication, or assignment submission when thousands of users access it simultaneously. This indicates inadequate performance and workload testing.

Testing Gaps

Possible gaps include:

- Insufficient load testing
- Lack of stress testing
- Unrealistic test data
- Limited concurrent-user testing
- Inadequate database testing
- Poor performance monitoring
- Missing recovery scenarios

Appropriate Testing

Load Testing: Measures behavior under expected user load.

Stress Testing: Determines how the system behaves beyond normal capacity.

Performance Testing: Measures response time, throughput, and resource consumption.

Recovery Testing: Determines whether the system can recover after overload or failure.

Example

If an online examination system works with 100 users but fails when 5,000 students submit answers simultaneously, the tester should monitor:

- Response time
- CPU utilization
- Memory consumption
- Database performance
- Network activity
- Error rate
- Transaction completion

Improvements

The testing team should create realistic peak-load scenarios and repeat them before every major release. Automated performance tests can also be used to simulate large numbers of users.

Conclusion: Performance problems during peak usage occur when real-world workload conditions are not adequately represented in testing. Load, stress, performance, and recovery testing should therefore form an important part of the testing strategy.

14. Early Involvement of Testers in the SDLC

Importance of Early Participation

Testers should not be introduced only after development has been completed. Their involvement from the requirement stage allows defects and ambiguities to be identified before implementation.

Role During Different Stages

Requirement Stage

Testers can:

- Review requirements
- Identify ambiguity
- Check testability
- Identify missing conditions

Design Stage

They can:

- Review workflows
- Identify integration risks
- Analyze data flow
- Identify possible failure conditions

Development Stage

They can:

- Prepare test data
- Review testability
- Support unit-test planning
- Identify automation opportunities

Testing Stage

They execute tests, report defects, perform retesting, and conduct regression testing.

Practical Example

Consider a banking fund-transfer requirement. An early tester may ask:

- What happens when the balance is insufficient?
- What happens when the beneficiary account is invalid?
- What happens if the network fails?
- What happens when the user submits twice?
- What happens when the transaction limit is exceeded?

These questions help the team clarify requirements before development.

Benefits

Early involvement results in:

- Earlier defect detection
- Lower rework cost
- Better requirements
- Improved testability
- Better risk identification
- Stronger collaboration

Conclusion: Early tester participation supports a shift-left approach to software quality. Testers contribute to defect prevention as well as defect detection, resulting in more reliable software.

15. Improving Incomplete Test Coverage

Understanding the Coverage Problem

A system may provide correct results for common scenarios but fail in less frequently tested situations. This usually indicates incomplete test coverage rather than complete software reliability.

Identifying Gaps

The testing team should compare requirements against existing test cases.

Coverage gaps may occur in:

- Functional requirements
- Negative scenarios
- Boundary conditions
- Error handling
- Integration paths
- Previously fixed defects
- Exceptional workflows

Improving Test Design

Formal techniques can be used to increase coverage.

Equivalence Partitioning identifies representative valid and invalid inputs.

Boundary Value Analysis focuses on minimum and maximum values.

Decision Tables test combinations of conditions.

State Transition Testing checks system behavior across different states.

Example

If an application accepts values from **1 to 100**, suitable tests include:

0, 1, 2, 99, 100, 101

Testing only 50 does not provide sufficient boundary coverage.

Recommended Actions

The team should:

1. Create a requirement traceability matrix.
2. Identify missing test cases.
3. Add negative and boundary scenarios.
4. Include historical defects in regression tests.
5. Automate repetitive high-priority cases.
6. Review coverage before release.

Conclusion: Complete coverage cannot guarantee defect-free software, but it increases confidence that important requirements and conditions have been properly tested. Systematic coverage analysis leads to stronger validation and reliability.

16. Improving Defect Reporting

Importance of Clear Reporting

Poor defect reporting can delay resolution because developers may not have enough information to reproduce the problem. A logistics application involving orders, inventory, warehouses, and deliveries requires precise defect communication.

Essential Information

A professional defect report should contain:

Defect ID – Unique identification number

Title – Short description of the problem

Environment – Browser, operating system, application version, etc.

Preconditions – Conditions required before execution

Steps to Reproduce – Exact sequence of actions

Expected Result – Correct behavior

Actual Result – Observed behavior

Severity and Priority – Business and technical importance

Evidence – Screenshots, logs, or videos when appropriate

Example

Instead of writing:

“Delivery date is wrong.”

A better report should specify the order number, delivery location, selected date, expected date, actual date, steps performed, and application version.

This allows the developer to reproduce the issue quickly.

Defect Lifecycle

A typical lifecycle is:

New → Assigned → In Progress → Fixed → Retest → Closed

If the problem continues during retesting, the defect can be reopened.

Benefits

Structured reporting improves:

- Reproducibility
- Communication
- Tracking
- Resolution speed
- Root-cause analysis
- Historical analysis

Conclusion: Clear defect reporting converts a tester's observation into useful technical information.

Standardized reports improve communication between testers and developers and support faster and more accurate defect resolution.

17. Process-Based Testing Across Modules

Need for Standardization

When different teams test different modules using different procedures, testing results may become inconsistent. A common process should therefore be established while allowing additional module-specific testing according to risk.

Common Testing Workflow

A standard process can be:

Requirement Analysis



Test Planning



Test Case Design



Environment Preparation



Test Execution



Defect Reporting



Retesting



Regression Testing

Every module should follow this basic workflow.

Standard Documentation

A common test-case format should contain:

- Test Case ID
- Objective
- Preconditions
- Test Data
- Steps
- Expected Result
- Actual Result
- Status

Defect reports should also follow one standard format.

Example

Consider an e-commerce system containing:

Login → Product Search → Shopping Cart → Payment

All modules can use the same fundamental testing process. However, the payment module should receive additional security, transaction, and integration testing because it carries greater risk.

Maintaining Consistency

Consistency can be supported through:

1. Standard operating procedures
2. Common test templates
3. Shared test-management tools
4. Common severity definitions
5. Test-case reviews
6. Shared regression suites
7. Automation guidelines

Conclusion: Process-based testing creates a consistent foundation across modules while still allowing additional testing based on risk. Standardization improves traceability, repeatability, and overall software quality.

18. Boundary Testing and Untested Edge Cases

Why Edge Cases Matter

Many software systems work correctly with normal inputs but fail when users enter values near limits or unusual combinations. These situations are known as edge cases.

Reasons for Missing Edge Cases

They may be missed because of:

- Incomplete requirements
- Limited test data
- Time constraints
- Focus on positive scenarios
- Lack of boundary analysis
- Insufficient historical defect analysis

Boundary Value Analysis

Boundary Value Analysis concentrates on values at and around the limits.

For example, if an ATM allows a maximum daily withdrawal of **₹50,000**, useful test cases include:

₹49,999 – Just below the limit

₹50,000 – Exact limit

₹50,001 – Just above the limit

These values are more useful than testing only an ordinary amount such as ₹10,000.

Other Edge Conditions

Testing should also include:

- Empty inputs
- Zero values
- Maximum field length
- Minimum values
- Maximum values
- Invalid dates
- Expired information
- Duplicate requests
- Network interruption

Improving Test Design

The team should identify boundaries directly from requirements and convert them into test cases. Important edge cases should also be included in regression testing after defects are fixed.

Conclusion: Boundary testing is an effective way to identify defects that remain hidden during normal testing. Systematically including edge cases improves reliability and reduces the risk of production failures.

19. Defect Documentation and Repository Management

Importance of Documentation

When defects are not properly documented, organizations lose important information about what went wrong, how the problem was reproduced, and how it was resolved. A structured repository provides a central record of defects throughout their lifecycle.

Information to Maintain

A defect repository should include:

Information	Purpose
Defect ID	Unique identification
Title	Quick understanding
Module	Identifies affected area
Description	Explains the problem
Severity	Measures impact
Priority	Determines urgency
Steps	Enables reproduction
Expected Result	Defines correct behavior
Actual Result	Records observed behavior
Root Cause	Supports prevention
Resolution	Records the fix
Status	Tracks lifecycle

Defect Lifecycle

A typical lifecycle is:

New → Assigned → In Progress → Fixed → Retest → Closed

If the defect still exists, it can be reopened.

Repository Analysis

The repository can be analyzed according to:

- Module
- Severity
- Priority
- Release
- Root cause
- Recurring defect type

For example, repeated payment defects may indicate weaknesses in transaction validation or integration.

Benefits

Proper documentation provides:

- Better traceability
- Faster reproduction
- Improved communication
- Historical knowledge
- Better root-cause analysis
- Stronger regression testing

Conclusion: A defect repository should serve as both a tracking system and a source of organizational learning. Proper documentation and analysis help teams understand recurring problems and improve future testing.

20. Redesigning Testing for Banking Transaction Processing

Need for a Redesigned Approach

Repeated failures in banking transaction processing indicate weaknesses in the existing testing strategy. Because transaction processing involves financial information, testing should cover functional correctness, integration, security, performance, boundaries, and recovery.

Proposed Testing Model

A suitable approach is:

Requirement Analysis → Risk Assessment → Test Design → Functional Testing → Integration Testing → Performance Testing → Security Testing → Regression Testing

Critical transaction requirements should be clearly identified before test design.

Important Test Conditions

The following conditions should be covered:

- Valid transaction
- Insufficient balance
- Invalid account
- Invalid authentication
- Transaction-limit violation
- Duplicate transaction
- Network failure
- Server timeout
- Transaction cancellation
- Recovery after failure

Example

Suppose Account A contains ₹20,000 and the customer transfers ₹5,000 to Account B.

The expected result is:

1. ₹5,000 is deducted from Account A.
2. ₹5,000 is credited to Account B.
3. Both balances are updated correctly.
4. Only one transaction record is created.
5. The transaction status is correctly displayed.
6. A confirmation is provided to the customer.

If the network fails during processing, the system must avoid duplicate or partial transactions.

Test Design Techniques

Equivalence Partitioning can separate valid and invalid transaction inputs.

Boundary Value Analysis can test transaction limits.

Decision Tables can evaluate combinations such as account status, balance, amount, and authorization. **State Transition Testing** can validate transaction states such as initiated, processing, successful, failed, and cancelled.

Defect Detection Improvements

The bank should introduce:

- Automated regression testing
- Risk-based test prioritization
- Integration testing
- Performance testing
- Security testing
- Network-failure testing
- Transaction recovery testing
- Historical defect analysis

Retesting and Regression

When a defect is fixed, the original scenario must first be retested. Regression testing should then be performed on related transaction functions to ensure that the correction has not introduced another problem.

Conclusion: Banking transaction processing requires a structured and risk-based testing approach.

Combining systematic test design, integration testing, boundary testing, performance and security validation, defect tracking, and regression testing can significantly improve transaction reliability and reduce repeated failures.